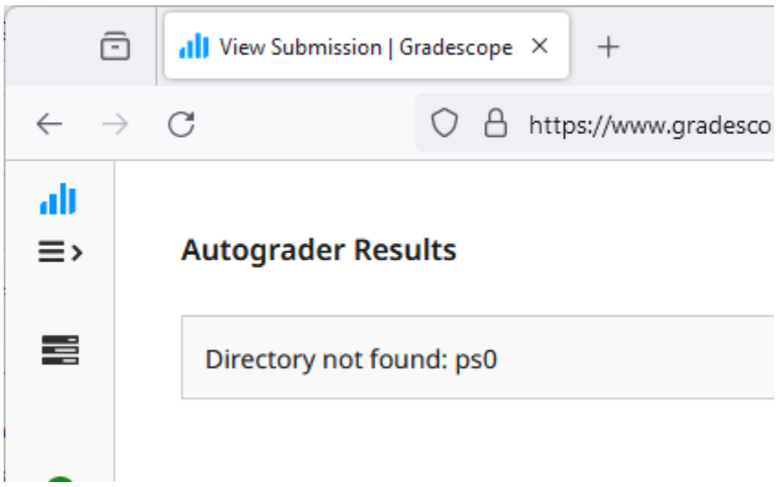
Submitting on Gradescope

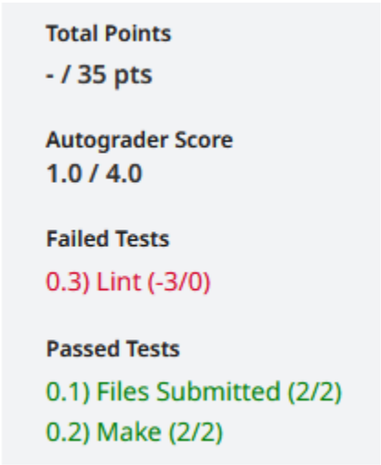
Making a Submission

Gradescope has a tutorial on how to submit assignments at `https://help.gradescope.com/article/ccbpppziu9-student-submit-work#submitting_code`. You can either upload a zip archive or link to a GitHub or Bitbucket repository. Make sure your repository is set to **private**. If your repository is public, there is a possibility that another student will copy from you and you will be penalized for helping someone cheat.

Your submission is required to be in a folder named *exactly* `ps0` (or similar for later assignments). Spelling matters and is case-sensitive, so directories named `PS0` will fail. In this case, the autograder will tell you "Directory not found: ps0". Fix it and resubmit. There is no penalty for making multiple submissions.



Afterwards, the autograder will run on your submission automatically. This will take a few minutes. When it finishes running, you will be able to see how you did on each of the autograder criteria (there are also manually graded parts that won't be done until after the submission deadline). On the right side, you will be able to see a summary of how you did. The Autograder Score is how well you did on the autograded criteria (including any late penalty). Below that will then be listed all of your failed tests (in red) and passed tests (in green). The individual test results *do not* include your late penalty. Above that, the Total Points is the sum of both the autograded and manual graded parts and will not be available until the TAs have graded the assignment.



In the main portion, you can see a more detailed listing of the autograder output. This will begin with the output of your `makefile` and checking your submission time. Gradescope checks your submission time against the deadline (including any extension if granted) and will compute how late you are and assign the late penalty automatically. Be sure to look at the Autograder Output box since if there are any compilation errors they will be displayed here.

Autograder Output

```
g++ --std=c++17 -Wall -Werror -pedantic -g -c main.cpp
g++ --std=c++17 -Wall -Werror -pedantic -g -o sfml-app main.o -lsfml-graphics -lsfm
```

On time

Below this will be a listing of the individual tests. Successful tests will be in green and failed tests will be in red. Unlike the summary, these tests appear in a fixed order rather than with the failed tests first. This order will be roughly the order that the instructors think you should tackle the tests. Each failed test will include information about what caused you to fail the test. You should use this information to fix your project and then make a new submission. You may make any number of submissions before the deadline at no penalty.

0.1) Files Submitted (2/2)

0.2) Make (2/2)

0.3) Lint (-3/0)

```
Test Failed: True is not false :
Skipping input '*.hpp': Can't open for reading
main.cpp:4:  Missing space before {  [whitespace/braces] [5]
main.cpp:18:  Missing space before ( in while(  [whitespace/parens] [5]
main.cpp:18:  Missing space before {  [whitespace/braces] [5]
Done processing main.cpp
Total errors found: 3
Lint suppression not allowed: main.cpp:10
```

In this example, we see that the student submitted all of the required files and their code compiled correctly, but there were some linting errors. We see that on line 18, they missed a space between `while` and `(` and on lines 4 and 18 they missed spaces before the `{` that starts a new block. Additionally, on line 10 they tried to suppress another error with `// NOLINT` which is not allowed in this class. Each type of linting error is worth -1 point.

Tips

Submit early, submit often

Gradescope will let you submit as many times as you want and there is no penalty applied for making multiple submissions. Whenever you accomplish something major, you should make a submission to check that you pass the tests you think you should pass. You can do this as soon as you have your initial setup complete to make sure that you have created all of the expected files in the correct directory. This will also let you see all of the tests and give you a better idea on exactly what you will be tested on.

You can continue to make submissions after the deadline (up to 1 week later for most assignments), but these will have a late penalty applied to them. Normally, your most recent submission is the one that is graded. If you want to use a different submission, you

can change which one is used by selecting "Submission History" at the bottom of the page and selecting your preferred submission.

Keep your code linted

If you wait until the very end to fix your linting errors, fixing them will feel overwhelming as you have dozens (or even hundreds) of errors that you must fix. If you lint as you go, you will only have a couple to fix at a time which will be much more manageable. It will also give you a better sense of what things the linter will require of you and you will naturally develop a better style (reducing future lint errors).

Remember that you don't have to make a submission to check lint errors. If you run

```
cpplint *.cpp *.hpp
```

you can see what kind of problems you will face. You should put this into your `makefile` so you can just run

```
make lint
```

Be sure to get the `CPPLINT.cfg` file so that you use the same settings as you will be graded on (some criteria are ignored by the autograder). This file needs to be placed in the same directory as the rest of your code. Note that the autograder will clobber your `CPPLINT.cfg` file so you can't cheat by modifying it to suppress certain types of errors. You also can't use the `// NOLINT` comment to suppress individual errors; doing so is worth -1 point *per instance*, versus the linting errors which are worth -1 point *per type*.

Write your tests first

You should write unit tests to verify that you code works as intended. For several of the projects, this will be required. You should write the tests *first*, before the code that they test. This helps ensure that you understand what your code is supposed to do, which is a key step to being able to write your project. It also give some certainty that your code works as expected since you will see them go from not compiling, to compiling but failing when you stub out the tested functions, to passing when you implement things. It also helps answer the question of whether your code works since you have your own tests to help answer that question. Running your own tests will be faster than submitting to the autograder, which can take a few minutes to run.

Additionally, for several assignments, there are discernment tests. For these tests, the autograder will provide several implementations and you will have to be able to differentiate between good implementations and bad implementations. To get credit, the bad implementation must fail at least one of your tests while the good implementation must pass all of them (so you can't have a simple "fail all" test). This will check both the validity of your tests (since you have to differentiate between good and bad implementations) and the breadth of your tests (since each bad implementation is broken in different ways). You can get credit for this even if your own implementation doesn't work.

Building a library

For most projects, you will be required to compile a static library (`.a` file). You can use the `ar` command to archive several object files into your library. For example, if you have object files `obj1.o`, `obj2.o`, and `obj3.o` and you want to create a library `MyLib.a`, you would run

```
ar rcs MyLib.a obj1.o obj2.o obj3.o
```

to create it. You could then link your program together using

```
g++ -o MyProg main.o MyLib.a
```

and `g++` would link your `main.o` file with the objects within your `MyLib.a` library to create `MyProg`. Note that you should not put any object files that contain `main` functions into the

library. This means that neither your `main.o` nor `test.o` objects should be placed into the library.

The autograder will link your library against the unit tests provided by the instructor to create the tests used by the autograder. If you accidentally add your `main.o` or `test.o` to the library, there will be a main redefinition error and none of the tests will pass. If you decide that you need to create additional classes beyond the ones that are required, you can put them in their own files and as long as they are included in the archive everything will be fine.